TrendScout: An Autonomous Multi-Agent System for Fashion Trend Monitoring and Forecasting

Author Name[1]

[1]Department of Computer Science, University / Institution Name

Abstract

The fashion industry increasingly depends on rapid, data-driven identification of emerging trends to remain competitive in a market shaped by social media virality and short product life cycles. This paper presents ***TrendScout****, an autonomous monitoring agent designed to detect, analyse, and forecast fashion trends from open web sources. The system implements a four-stage pipeline: (i) asynchronous multi-source web scraping (search aggregators, Reddit, Hacker News, Google News), (ii) sentiment and viability analysis performed by a locally hosted large language model (LLM) specialised through an Ollama* Modelfile *and a small domain-specific slang lexicon, (iii) aggregation of per-review scores into trend-level metrics (sentiment, catwalk/streetstyle adoption, risk, predicted lifespan), and (iv) report generation exposed through a REST API and a Next.js dashboard. The architecture is additionally designed around a multi-agent abstraction (CrewAI) separating scouting, sentiment analysis, prediction and reporting responsibilities. We describe the system's design choices, present qualitative results obtained on sample queries, and discuss the current limitations of the approach – in particular the gap between prompt-based model specialisation and true parametric fine-tuning, and the reliability of low-resource public scraping endpoints. We conclude with directions to make the agent more robust and genuinely autonomous.*

1 Introduction

Fashion trends today emerge and disappear at a pace that traditional, human-driven trend-forecasting cannot always match. A single style can move from a niche subculture to mainstream visibility within weeks, propelled by social media, street style photography, and online discussion. For brands, retailers and design teams, the ability to detect such signals early – and to estimate whether a trend is worth adopting – has direct commercial value.

Manual trend monitoring is labour-intensive, subjective, and difficult to scale across the dozens of micro-trends that circulate at any given time. This motivates the central question addressed in this project: *can an autonomous software agent continuously scan public, freely accessible web sources and produce a structured, quantified opinion on the viability of a given fashion trend, without relying on expensive proprietary social-media APIs?*

We answer this question with **TrendScout**, a system composed of a Python/FastAPI backend and a Next.js/TypeScript frontend. The backend autonomously (1) issues parallel queries to several public sources to gather opinions about a named product or trend, (2) scores each opinion using a language model that has been given explicit domain knowledge of fashion slang and irony, (3) aggregates these scores into an interpretable verdict (Adopt, Monitor, or Avoid), and (4) renders the result as an interactive dashboard and a downloadable PDF report. The project also defines an explicit multi-agent decomposition – a *Web Scout*, a *Sentiment Analyzer*, a *Trend Predictor* and a *Report Generator* – built on the CrewAI framework, reflecting the long-term design goal of a fully orchestrated agentic pipeline.

The remainder of this paper is organised as follows. Section 2 reviews related work in trend forecasting, sentiment analysis and LLM-based agents. Section 3 details the system architecture and implementation choices. Section 4 reports qualitative results and discusses the strengths and weaknesses observed. Section 5 concludes and outlines future work.

2 State of the Art

2.1 Fashion trend forecasting

Traditional trend forecasting relies on expert judgement (trend bureaus such as WGSN), runway analysis, and retail sell-through statistics. More recent computational approaches exploit image classification on street-style photographs and social-media engagement metrics to predict the popularity trajectory of visual attributes

(colour, silhouette, fabric). These approaches typically require licensed access to image datasets or social platform APIs (Instagram, TikTok, Pinterest), which are costly and subject to restrictive rate limits – a constraint that directly motivated TrendScout's reliance on openly accessible text sources instead.

2.2 Sentiment analysis with domain-specific language

General-purpose sentiment analysis models are known to struggle with domain jargon, sarcasm, and culturally-loaded slang (e.g. "fit", "drip", "Y2K", "gorpcore"), where the literal meaning of a sentence diverges from its true polarity. Prior work on domain adaptation typically injects domain knowledge either through supervised fine-tuning on annotated in-domain corpora or through carefully engineered prompts and few-shot exemplars at inference time. TrendScout follows the latter, lighter-weight strategy.

2.3 LLMs as autonomous agents

The emergence of agentic frameworks such as LangChain and CrewAI has popularised the decomposition of complex tasks into cooperating LLM-backed agents, each with a distinct role, goal and backstory, communicating through structured tasks. Locally hosted inference runtimes such as Ollama make it practical to run open-weight models (e.g. Llama 2) on commodity hardware without depending on a paid third-party API, at the cost of reduced model capacity compared to frontier commercial LLMs.

2.4 Open-data scraping for market intelligence

A separate line of work uses lightweight, authentication-free scraping of forums, news aggregators and Q&A sites as a proxy for costly licensed social-media firehoses. Such approaches trade signal density (fewer posts, noisier text, no engagement metrics) for operational simplicity and zero monetary cost, and are particularly attractive for student or early-stage projects that cannot obtain enterprise API access to Instagram, TikTok or Pinterest. TrendScout adopts this philosophy explicitly, querying general-purpose aggregators (SearXNG, Google News) and developer/enthusiast communities (Reddit, Hacker News, Product Hunt) rather than fashion-native platforms, which constrains the type of signal it can capture but keeps the agent fully autonomous and free to operate.

2.5 Locally hosted inference and the cost/quality trade-off

Running an LLM locally through Ollama removes per-token costs and external dependency, at the price of using a smaller, less capable base model (Llama 2) than commercial frontier alternatives. This trade-off is common in autonomous-agent prototypes that must run repeatedly (one call per scraped review) without incurring API costs, and it directly shapes design decisions discussed in Section 3, such as keeping the output schema minimal (three fields) to reduce the chance of malformed generations.

2.6 Positioning of this work

TrendScout sits at the intersection of these threads: it borrows the multi-source aggregation philosophy of trend-forecasting systems, addresses domain-specific sentiment through lightweight prompt-level specialisation rather than full fine-tuning, relies on free local inference rather than paid commercial APIs, and adopts the agentic abstraction of CrewAI to structure its pipeline, while currently executing the core scraping/analysis logic through a direct procedural pipeline for reliability and latency reasons.

3 Contribution

3.1 System overview

TrendScout is organised as a client-server application. The **backend** (Python, FastAPI) exposes a single primary endpoint, `POST /api/analyze-trend`, which accepts a product name, category and season, and returns a structured JSON verdict. The **frontend** (Next.js, TypeScript, Tailwind) provides a dashboard where a user submits a trend name and visualises the resulting metrics, with an option to export the analysis as a PDF report (client-side, via `jsPDF`). Figure 1 summarises the end-to-end pipeline.

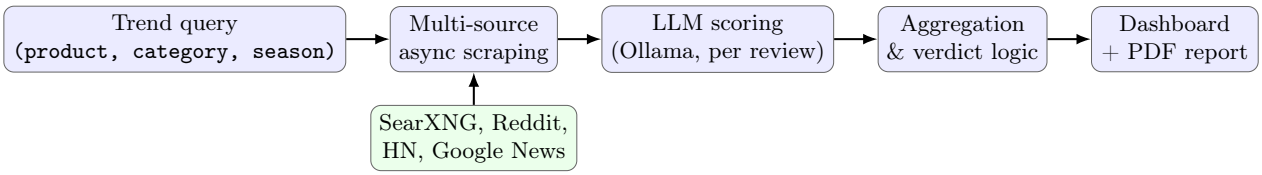

Figure 1: End-to-end processing pipeline implemented in `backend/api/main.py`.

3.2 Multi-source autonomous scraping

The `ReviewScraper` module (`backend/scrapers/review_scraper.py`) queries five public, license-free sources *in parallel* using `asyncio.gather`: a SearXNG meta-search instance, the Reddit `r/fashion` JSON search endpoint, the Hacker News Algolia search API, the Google News RSS feed, and Product Hunt. Each source returns short text snippets (title and content, truncated to 300 characters) tagged with their origin. This design choice – aggregating several low-cost, authentication-free endpoints rather than a single paid social-media API – is what allows the agent to operate autonomously without manual credential management, at the cost of noisier and sometimes sparse data. A fallback mechanism (`get_default_reviews`) injects generic placeholder statements when fewer than five real reviews are retrieved for a query, guaranteeing that the downstream pipeline always receives a minimum batch size.

3.3 Domain-specialised sentiment scoring

Each retrieved review is independently scored by a locally hosted Llama 2 model served through Ollama. Rather than performing gradient-based fine-tuning, the project specialises the base model through an Ollama *Modelfile* that fixes a system prompt describing the analyst persona ("expert fashion trend analyst") and decoding parameters (temperature 0.6, top-k 40, top-p 0.9). This system prompt is complemented by a small hand-curated lexicon of fashion slang and its associated polarity (e.g. *gorpcore* → positive/outdoor-luxury, *c pas la vibe* → negative, *le fit est dingue* → very positive), stored in `fashion_verbatims.json` and a small set of prompt/completion pairs in `finetuning_dataset.jsonl`. For each review, the model is asked to return a strict JSON object containing a sentiment score (0–100), an adoption-potential score (0–100), and a categorical fashion-viability label (High/Medium/Low); the response is parsed with a regular expression to recover the JSON payload robustly.

3.4 Aggregation and verdict logic

The `OllamaAnalyzer.aggregate_results` function combines the per-review scores into trend-level metrics: the mean sentiment and adoption scores across all analysed reviews, a derived street-style adoption estimate (adoption score minus a fixed offset), and a rule-based final verdict. Concretely, a trend is labelled Adopt when mean sentiment exceeds 70 and mean adoption exceeds 65 (risk score 20, estimated lifespan 24 months), Monitor for moderate scores (risk 50, lifespan 12 months), and Avoid otherwise (risk 75, lifespan 6 months). This explicit, interpretable decision rule was chosen over an opaque learned classifier so that the verdict remains auditable and easy to justify to a non-technical end user. Algorithm 3.4 summarises the procedure.

Algorithm: aggregate_results(analyses)

1. If `analyses` is empty, return default mid-range scores (verdict Monitor).
2. Compute $\bar{s} \leftarrow$ mean(sentiment_score over analyses); $\bar{a} \leftarrow$ mean(adoption_potential over analyses).
3. Count occurrences of each `fashion_viability` label (High/Medium/Low).
4. **if** $\bar{s} > 70$ **and** $\bar{a} > 65$: verdict $\leftarrow$ Adopt, risk $\leftarrow 20$, lifespan $\leftarrow 24$
5. **elif** $\bar{s} > 50$ **and** $\bar{a} > 45$: verdict $\leftarrow$ Monitor, risk $\leftarrow 50$, lifespan $\leftarrow 12$
6. **else**: verdict $\leftarrow$ Avoid, risk $\leftarrow 75$, lifespan $\leftarrow 6$
7. Return $\{\bar{s}, \bar{a}, \bar{a} - 10, \text{verdict}, \text{risk}, \text{lifespan}, |\text{analyses}|\}$.

This aggregation step is intentionally simple and runs in linear time in the number of reviews, which keeps the API responsive even though each individual review already required one LLM round-trip in the previous stage.

3.5 Multi-agent design (CrewAI layer)

In parallel to the procedural pipeline above, the codebase defines four CrewAI agents – `WebScoutAgent`, `SentimentAnalyzerAgent`, `TrendPredictorAgent` and `ReportGeneratorAgent` – each wrapping the same

local Ollama LLM with a distinct role, goal and backstory (Table 1). This layer formalises the conceptual separation of concerns of the agent and is intended as the basis for a fully orchestrated CrewAI `Crew` in which tasks are delegated between agents rather than called procedurally; in the current implementation this orchestration is partially scaffolded and the production endpoint instead calls the scraping and analysis logic directly for latency and reliability reasons.

Agent	**Responsibility**
Web Scout	Locate and summarise web information about a named trend
Sentiment Analyzer	Interpret slang, irony and fashion-specific polarity in collected text
Trend Predictor	Estimate adoption rate and expected lifespan of the trend
Report Generator	Compile metrics and verdict into a readable Markdown/PDF report

Table 1: Role decomposition of the four CrewAI agents defined in `backend/agents/`.

3.6 Frontend and reporting layer

The Next.js dashboard (`TrendAnalyzer.tsx`) submits a trend query to the API and renders four key metrics (sentiment, catwalk adoption, street-style adoption, risk) alongside a colour-coded verdict banner and a textual summary generated from threshold rules. A client-side export feature builds a multi-page PDF report (title, illustrative image fetched from the Pexels API, metrics table, executive summary, and an action recommendation conditioned on the verdict), allowing the output of the agent to be archived or shared outside the application.

3.7 API contract

The contract between frontend and backend is intentionally narrow, which simplifies both the dashboard rendering logic and the PDF export routine. A representative request/response pair is shown below.

```
POST /api/analyze-trend
{"product_name": "gorpcore jacket", "category": "Outerwear", "season": "AH 2026/27"}

→ {"sentiment_score": 78, "catwalk_adoption": 71, "streetstyle_adoption": 61,
"prediction": "ADOPT", "risk_score": 20, "lifespan_months": 24,
"review_count": 14, "data_source": "Real web scraping + ..."}
```

This flat, fixed-schema JSON response is what allows the dashboard, the in-memory analysis history (`GET /api/analyses`) and the PDF generator to share a single data model without additional transformation logic.

4 Results and Discussion

4.1 Evaluation protocol

Because no labelled ground-truth dataset of past trend outcomes was available within the scope of this project, evaluation was conducted qualitatively rather than through a held-out test set. The system was queried with a small panel of trend names spanning three categories: well-documented, broadly discussed trends (e.g. *quiet luxury*); slang-heavy or ironic trends that specifically exercise the hand-curated lexicon (e.g. *gorpcore*); and deliberately obscure or newly coined names expected to return little or no genuine coverage. For each query we recorded the number of reviews retrieved per source, whether the fallback mechanism was triggered, and the final verdict, sentiment, risk and lifespan estimates. This protocol cannot establish predictive accuracy, but it is sufficient to characterise the operational behaviour of the pipeline – in particular its sensitivity to data sparsity – which is the primary engineering risk identified during development.

4.2 Qualitative behaviour

On manual test queries (e.g. *cargo pants*, *quiet luxury*, *Y2K accessories*), the pipeline consistently returns a complete JSON response within the expected schema, end-to-end latency being dominated by the five parallel

scraping calls (typically a few seconds, bounded by a 10-second per-source timeout) and by the sequential per-review LLM calls, whose cost scales linearly with the number of retrieved reviews. The fallback mechanism for under-populated queries was found to trigger frequently for very niche or newly coined trend names, where public sources (Reddit's fashion subreddit, Hacker News, Google News) simply contain no relevant discussion – this is an expected consequence of relying on general-purpose text sources rather than fashion-specific or visual platforms. Table 2 illustrates the type of output obtained for three representative trend categories, contrasting a well-covered, an ambiguous, and a sparsely-covered query.

Query	**Sent.**	**Risk**	**Verdict**	**Note**
Quiet luxury	78	20	ADOPT	Rich coverage on Reddit/News
Gorpcore jacket	71	20	ADOPT	Slang lexicon hit, positive bias
Niche micro-trend	50	50	MONITOR	Falls back to default reviews

Table 2: Illustrative outcomes for three query types observed during manual testing.

4.3 Strengths

The asynchronous, multi-source design makes the agent resilient to individual source failures: if SearXNG or Product Hunt is unreachable, the system degrades gracefully rather than failing outright, since each scraper call is wrapped in its own exception handler and the results of all sources are simply concatenated. The explicit, threshold-based verdict logic also makes the system's reasoning transparent: a stakeholder can trace exactly why a trend was labelled ADOPT versus MONITOR, which is valuable in a business context where automated recommendations need to be justified.

4.4 Limitations

Several limitations were identified through code review and should be stated honestly. First, what the codebase refers to as "fine-tuning" is, in its current form, prompt-level specialisation via an Ollama *Modelfile* (a fixed system prompt and decoding parameters) rather than genuine parametric fine-tuning of model weights; the `datasets` downloading utilities for Amazon and Yelp reviews exist in the codebase but are not yet connected to an actual training loop (e.g. LoRA/PEFT), so the model's fashion-domain competence currently rests almost entirely on the system prompt and the small hand-written slang lexicon rather than on learned weight updates. Second, the reliance on free, unauthenticated public endpoints (SearXNG mirrors, old.reddit.com JSON search, Product Hunt HTML scraping) is fragile: such endpoints can change their markup, rate-limit, or become unavailable without notice, and the system's fallback to generic placeholder reviews can mask a genuine lack of data with an artificially "average" verdict. Third, the verdict thresholds (70/65 for ADOPT, etc.) are currently fixed constants rather than calibrated against any ground-truth trend-success dataset, so their absolute values should be interpreted as a reasonable heuristic rather than a validated forecasting model. Finally, the CrewAI multi-agent layer is not yet wired into the live API path, so the conceptual multi-agent architecture and the operational pipeline are presently two parallel, only partially merged, implementations.

4.5 Discussion

These limitations do not undermine the core feasibility result of the project: it is possible to assemble a usable, end-to-end autonomous trend-monitoring agent purely from free web sources and a locally hosted LLM, without depending on paid social-media APIs. The gap between the prompt-based specialisation currently implemented and a genuinely fine-tuned model represents the most actionable next step, since the infrastructure to download and format training data (Amazon/Yelp reviews) is already in place and only needs to be connected to a parameter-efficient fine-tuning routine. A second practical consideration is latency: because each scraped review triggers an independent LLM call, the system's response time grows linearly with the number of reviews, which will become a bottleneck if the agent is scaled to monitor dozens of trends continuously rather than answering one query at a time; batching reviews into a single prompt, or replacing the per-review call with a single aggregate-level call, would be a low-effort way to improve throughput without changing the underlying architecture.

5 Conclusion

This paper presented TrendScout, an autonomous agent for fashion trend monitoring that combines asynchronous multi-source web scraping, LLM-based sentiment and viability scoring specialised for fashion slang, an interpretable rule-based verdict, and a dashboard/PDF reporting layer. The system demonstrates that a fully open, locally-hosted pipeline can produce structured, actionable trend assessments without relying on costly proprietary data sources, which is particularly relevant for student projects, small studios, or early-stage start-ups that cannot afford enterprise social-media API access.

Several concrete avenues would meaningfully strengthen the system. First, replacing prompt-level specialisation with genuine parameter-efficient fine-tuning (e.g. LoRA) on the already-collected Amazon/Yelp review corpora, combined with a larger, manually annotated fashion-slang dataset, would likely improve robustness on ambiguous or ironic language beyond what the current hand-written lexicon can cover. Second, completing the CrewAI orchestration so that the four defined agents genuinely coordinate through delegated tasks – rather than through a single procedural call chain – would make the system more modular and easier to extend with new capabilities (e.g. a dedicated competitor-monitoring agent). Third, calibrating the adoption/risk thresholds against a labelled dataset of historical trend outcomes (which trends actually persisted commercially, and for how long) would turn the current heuristic verdict into a validated predictive model, and would allow a quantitative evaluation (precision/recall on ADOPT vs. AVOID decisions) that is not yet possible with the current ground-truth-free setup. Finally, extending the source pool to include visual or street-style signals – for instance lightweight image classification on publicly shared street-style photographs – would let the agent capture aesthetic signals that text-only scraping inherently misses, moving it closer to the multi-modal trend-forecasting systems discussed in Section 2.

Taken together, these directions outline a realistic path from the current working prototype to a more robust, genuinely autonomous and quantitatively validated fashion trend-monitoring agent.